RESOLV-HQ

TOOL FAILURE & AUTHORIZATION

TEST EVIDENCE — WEEK 4

Date: 25 September 2026

Requirement: Week 4 — Tests for missing parameters, unauthorized requests, unavailable services, and unexpected tool responses

Tools under test: docs/tool-catalogue.txt

Purpose. This document records executed evidence for the RESOLV-HQ agent-tool failure and authorization tests. The tests exercise the actual implementation rather than a reimplementation or hypothetical behaviour.

1. Method

executeAgentTool() and the four tool functions in resolv-hq-backend/src/lib/agent-tools.ts are pure. They take a context object (knowledge, activeRequests, account) already scoped to one caller and produce a plain-text result, without a database or network call inside the tool layer itself.

This allows the tools to be exercised directly outside a running server while following the same code path used in production: react-agent.ts's Act step calls the same executeAgentTool() implementation.

A throwaway script was executed with npx tsx against the real source file. It covered the five recorded test cases below and was deleted afterward; it was never committed.

2. Case 1 — Missing Required Parameter

Setup: Call draft_escalation_ticket with no summary, simulating a tool call that reached the executor without its required argument. This is a defense-in-depth test because the JSON schema already marks summary as required.

Command:

```
executeAgentTool("draft_escalation_ticket", {}, customerA)
```

Actual captured output:

```
Cannot draft a ticket without a summary of the issue.
```

Assessment: PASSED. No hollow ticket is drafted; the failure is explicit and would surface to the caller as a request for more detail rather than a silent empty draft.

3. Case 2 — Unauthorized / Cross-Customer Access

Setup: Two distinct caller contexts were created: customer A (Kampala, one open request, “Login failure after reset”) and customer B (Jinja, one resolved request, “Billing question”). The two caller-bound tools were each called once for both contexts.

Commands:

```
executeAgentTool("account_status_lookup", {}, customerA)
executeAgentTool("account_status_lookup", {}, customerB)
executeAgentTool("outage_status_checker", {}, customerA)
executeAgentTool("outage_status_checker", {}, customerB)
```

Actual captured output:

```
=== account_status_lookup, context = customer A ===
Role: customer
Status: active
Location: Kampala, Uganda
Member since: 2025-01-10

=== account_status_lookup, context = customer B ===
Role: customer
Status: active
Location: Jinja, Uganda
Member since: 2024-06-02

=== outage_status_checker, context = customer A ===
- "Login failure after reset": open

=== outage_status_checker, context = customer B ===
- "Billing question": resolved
```

Assessment: PASSED. Each invocation returned only the data for the context supplied to it. Neither tool exposes a user-ID, customer-ID, request-ID, or account-selector argument. Therefore, a model cannot construct a tool call that selects another caller's data.

In production, chat.ts builds the caller context using loadAccountInput(userId, role), with the account resolved server-side from req.user rather than supplied by the client. The authorization boundary is therefore structural: no selector exists at the tool interface.

4. Case 3 — Unavailable / Unrecognized Tool

Setup: Call the executor with a tool name that is not registered, simulating a model hallucinating a tool or attempting to use a retired tool.

Command:

```
executeAgentTool("refund_processor", {}, customerA)
```

Actual captured output:

```
Unknown tool "refund_processor".
```

Assessment: PASSED. The executor produces a clear, catchable message rather than throwing an exception or silently doing nothing. refund_processor is also representative of an operation the system deliberately does not expose under the AI boundary.

5. Case 4 — Unexpected Tool Response: Ungrounded Query

Setup: Query the knowledge base for a topic with zero keyword overlap with the test corpus.

Command:

```
executeAgentTool("search_knowledge_base", { query: "vegetable garden watering schedule" }, customerA)
```

Actual captured output:

```
No matching knowledge base articles found.
```

Assessment: PASSED. The result conforms to the prompt specification: when the knowledge base does not cover the question, the tool reports that fact instead of encouraging an unsupported guess.

6. Case 5 — Schema Strictness

Setup: Read AGENT_TOOL_DEFINITIONS directly and confirm that every registered tool declares an explicit required array and additionalProperties: false.

Actual captured output:

```
[
  { "name": "search_knowledge_base", "required": ["query"], "additionalProperties": false },
  { "name": "account_status_lookup", "required": [], "additionalProperties": false },
  { "name": "outage_status_checker", "required": [], "additionalProperties": false },
  { "name": "draft_escalation_ticket", "required": ["summary"], "additionalProperties": false }
]
```

Assessment: PASSED. All four tool schemas are strict. Required inputs are explicit and undeclared arguments are not part of the permitted schema.

7. Genuine Finding — Keyword-Overlap False Match

Status: GENUINE LIMITATION — LOG FOR WEEK 7 FAILURE CATALOGUE.

While preparing Case 4, an initial test used the query “how do I reset a satellite modem” against the same one-article corpus. The corpus article concerned password resets, yet the tool returned it because both the query and article contained the word “reset.”

Observed output:

```
=== search_knowledge_base("how do I reset a satellite modem") ===
### Account Access Troubleshooting Guide
If a login failure persists after a password reset...
```

Root cause: scoreMatch() in agent-tools.ts ranks results using raw word overlap between the query and article title/content. There is no relevance threshold beyond score > 0. Consequently, one shared word can be sufficient to surface an otherwise unrelated article.

Why this matters. This is a reproducible retrieval weakness rather than a hypothetical failure. It can produce a confident-looking grounding result for an unrelated question. The finding should be carried into the Week 3 RAG evaluation and the compiled Week 7 Failure Catalogue.

Recommended catalogue wording: “Keyword-overlap retrieval can surface a coincidentally matching article for an unrelated query instead of correctly reporting no grounding, when the query happens to share a common word with an article's content.”

8. Test Summary

Case	What it tests	Result
1	Missing required parameter	PASSED
2	Unauthorized / cross-customer access	PASSED — structural; no selector argument
3	Unavailable / unrecognized tool	PASSED
4	Unexpected response — ungrounded query	PASSED
5	Schema strictness	PASSED
—	Keyword-overlap false-match	GENUINE LIMITATION — logged for Week 7

9. Requirement Coverage

The executed evidence covers all required Week 4 test categories specified in the brief:

- Missing required parameter — Case 1.
- Unauthorized / cross-customer request — Case 2.
- Unavailable or unrecognized service/tool — Case 3.
- Unexpected tool response / ungrounded retrieval — Case 4.
- Additional structural schema validation — Case 5.

The evidence was captured against the actual tool implementation in resolv-hq-backend/src/lib/agent-tools.ts, rather than an independent test reimplementation.

10. Evidence and Source Files

- resolv-hq-backend/src/lib/agent-tools.ts — executeAgentTool(), tool functions, scoring and schemas.
- resolv-hq-backend/src/lib/react-agent.ts — production ReAct Act-step integration.
- resolv-hq-backend/src/routes/chat.ts — server-side caller-context loading and authorization boundary.
- docs/tool-catalogue.txt — catalogue of the four tools under test.
- docs/function-calling-schemas.md — function-calling design rationale and domain mapping.
- docs/ai-boundary-matrix.md — AI tool boundary and approval rules.
- docs/evaluation-table.md — recommended location for the RAG-specific retrieval case.